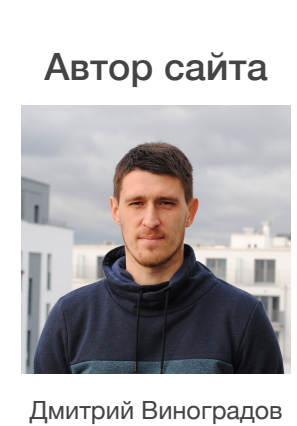
Language

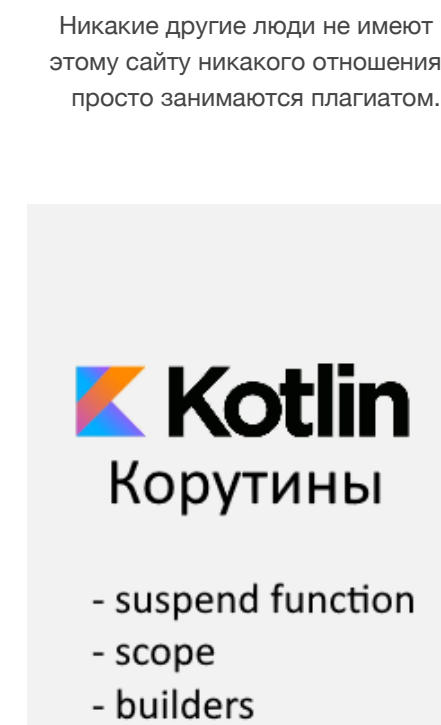
Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Kotlin
Корутины
- suspend function
- scope
- builders
- exceptions
- flow

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 067 подписчиков
Подписаться

Поддержка проекта

Яндекс
410011180491924
WebMoney
R248743991365
Z551306702056
Paypal

Урок 37. User Input. Consumed

28 марта 2024

В этом уроке разбираемся, зачем нужен статус consumed.

На прошлых уроках мы рассматривали input события на примере одного Box. Но это слишком просто. В реальном приложении у нас обычно целая иерархия UI компонентов. На экране они накладываются друг на друга - как слои.

В качестве примера можно рассмотреть приложение какой-либо соц.сети. Есть экран с лентой постов. У каждого поста внизу есть набор доступных действий - Like, Favourites, Share и т.д. Даже в самом простом случае это будет несколько слоев. Корневой Column, в нем список - LazyColumn, в списке каждый элемент - это Column, в котором есть Row с набором иконок-действий.

В итоге под каждой иконкой оказывается куча слоев - UI компонентов. Когда мы кликаем на иконку, клик пойдет через все эти компоненты. И им надо будет между собой разобраться, кто из них будет реагировать на событие.

Если мы жмем, например, на Like, то пост должен проигнорировать этот клик. Т.е. должен поставиться лайк, а не открыться пост.

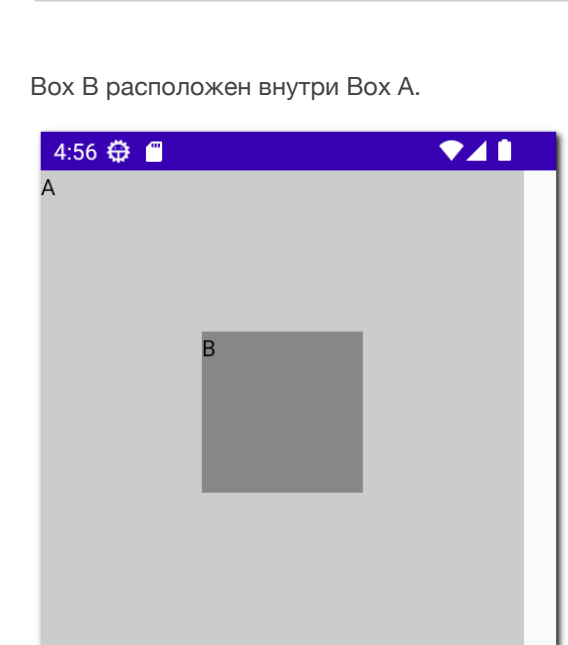
Или более сложный случай - когда мы ставим палец на пост. В этот момент у нас появляются варианты развития событий. Если мы отпустим палец, это будет клик, за который отвечает пост. Если мы начнем двигать палец вниз, то это уже скролл, за который отвечает список.

В Compose есть механизмы, которые позволяют компонентам договариваться между собой, чтобы корректно обрабатывать события.

Рассмотрим эти механизмы на примере двух Box:

```
@Composable
fun HomeScreen() {
   val logs = remember { mutableStateListOf<String>() }
   Column {
       Box(modifier = Modifier.size(300.dp)
           .background(Color.LightGray)
           .inputA(logs)
       ) {
           Text("A")
           Box(modifier = Modifier.size(100.dp)
               .background(Color.Gray)
               .align(Alignment.Center)
               .inputB(logs)
           ) {
               Text("B")
           }
       }
       Text(
           text = logs.takeLast(12).joinToString("\n"),
           fontSize = 14.sp
       )
   }
}
```

Box B расположен внутри Box A.



Для каждого из Box создаем отдельные Modifier методы: inputA и inputB. Это не какое-то требование, просто так будет удобнее менять код обработки жестов по ходу урока. Основной код примера меняться не будет.

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {

   }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {

   }
}
```

Слои

С точки зрения слоев - Box B расположен поверх Box A. И как я уже говорил, событие пойдет насквозь, через слои. Т.е. когда мы кликаем на B, это событие придет не только в B, но и в A. Давайте проверим это.

Для обоих Box добавляем awaitPointerEvent, который просто ловит и выводит в лог все события:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           while (true) {
               val event = awaitPointerEvent()
               logs.add("B ${event.type}")
           }
       }
   }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           while (true) {
               val event = awaitPointerEvent()
               logs.add("A ${event.type}")
           }
       }
   }
}
```

Запускаем, кликаем на B

Видим, что каждый Box поймал по два события.

Обратите внимание на порядок. Каждое событие приходит сначала в B, потом в A. Т.е. событие идет сквозь слои - от верхнего к нижнему.

На самом деле там все немного сложнее. Событие бегает туда-сюда несколько раз. Об этом чуть позже еще поговорим. А пока будем рассматривать только один прогон - основной.

clickable

Мы увидели, что события, которые позволяют определить клик, приходят и в A и в B. Значит, если мы повесим на каждый Box по clickable, то он сработает для обоих Box?

Проверим:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.clickable { logs.add("B click") }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.clickable { logs.add("A click") }
}
```

Запускаем и кликаем на B и на A

Когда мы кликаем на B, его clickable ловит клик. А вот clickable A при этом не срабатывает. Это стандартное поведение. Верхний слой взял клик на себя.

Чтобы проверить, что clickable A вообще работает, мы кликаем на A и видим, что все ок.

Осталось понять, откуда clickable A знает, что не надо срабатывать при клике на B?

На первый взгляд кажется, что clickable B обрабатывает события так, что они потом не приходят в A. Давайте проверим эту гипотезу.

Оставляем clickable на B, а для A снова сами ловим события и выводим их в лог:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.clickable { logs.add("B click") }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           while (true) {
               val event = awaitPointerEvent()
               logs.add("A ${event.type}")
           }
       }
   }
}
```

Кликаем на B

События приходят в A. Это значит, что clickable B не останавливает события на их пути к A. Но он меняет их статус.

В примере с двумя clickable происходило следующее. clickable B ловил событие и помечал его, как обработанное. Далее это событие шло в clickable A, который проверял статус события. Поняв, что событие уже обработано, clickable A просто игнорировал его. Поэтому мы и не получали клик на A.

Давайте выведем в лог этот статус в обработке событий в A:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.clickable { logs.add("B click") }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           while (true) {
               val event = awaitPointerEvent()
               logs.add("A ${event.type} consumed = ${event.changes.firstOrNull()?.isConsumed}")
           }
       }
   }
}
```

У PointerInputChange есть поле isConsumed, которое позволяет определить, что касание уже обработано.

Кликаем сначала на B, потом на A

Видим, что событие, которое прошло через clickable B, пришло в A со статусом consumed. А если кликать на A, то события приходят не обработанные.

consumed

Чтобы лучше понять эту тему, давайте попробуем внедрить consumed статус в наш простой обработчик кликов, который мы делали на прошлых уроках.

Добавим его в оба Box.

Box B:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitPointerEventScope {
           while (true) {
               var event1: PointerEvent
               do {
                   event1 = awaitPointerEvent()
               } while (event1.type != PointerEventType.Press)
               logs.add("B press")

               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release)
               logs.add("B click")
           }
       }
   }
}
```

Box A:

```
fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitPointerEventScope {
           while (true) {
               var event1: PointerEvent
               do {
                   event1 = awaitPointerEvent()
               } while (event1.type != PointerEventType.Press)
               logs.add("A press")

               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release)
               logs.add("A click")
           }
       }
   }
}
```

Сначала мы ждем событие Press, потом - Release. После этого считаем, что был совершен клик.

Это работало, когда у нас был один Box. Проверим, как работает с двумя.

Кликаем на B

Наш обработчик считает, что клик был и на B, и на A. Это неправильно.

Чтобы это поправить, надо работать с consumed.

Для начала сделаем так, чтобы обработчик B ставил consumed статус

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitPointerEventScope {
           while (true) {
               var event1: PointerEvent
               do {
                   event1 = awaitPointerEvent()
               } while (event1.type != PointerEventType.Press)
               logs.add("B press")
               event1.changes.first().consume()

               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release)
               logs.add("B click")
               event2.changes.first().consume()
           }
       }
   }
}
```

После того как поймали каждое событие (Press и Release), вызываем consume() у его касания.

Но этого недостаточно. Обработчик в A должен будет проверить этот статус.

```
fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.pointerInput(Unit) {
       awaitPointerEventScope {
           while (true) {
               var event1: PointerEvent
               do {
                   event1 = awaitPointerEvent()
               } while (event1.type != PointerEventType.Press || event1.changes.first().isConsumed)
               logs.add("A press")

               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release || event2.changes.first().isConsume
               logs.add("A click")
           }
       }
   }
}
```

Дополняем условия циклов. Первый цикл игнорирует все события, которые consumed или не Press. Т.е. он ждет не обработанный Press. Аналогично второй цикл ждет необработанный Release.

Проверяем клики на B

Все работает, как должно. Клик на B не приводит к клику A.

Обработчик кликов B у нас ставит статус consumed, а обработчик кликов A учитывает этот статус. Смержим логику из двух этих обработчиков в один, который будет и ставить статус и учитывать его. Такой обработчик будет универсальным, и его можно будет использовать в обоих Box.

Назовем его myClickable:

```
fun Modifier.myClickable(onClick: () -> Unit): Modifier {
   return this.pointerInput(Unit) {
       awaitPointerEventScope {
           while (true) {
               var event1: PointerEvent
               do {
                   event1 = awaitPointerEvent()
               } while (event1.type != PointerEventType.Press || event1.changes.first().isConsumed)
               event1.changes.first().consume()

               var event2: PointerEvent
               do {
                   event2 = awaitPointerEvent()
               } while (event2.type != PointerEventType.Release || event2.changes.first().isConsume
               event2.changes.first().consume()


               onClick()
           }
       }
   }
}
```

На вход даем ему лямбду, которая будет вызываться. Все как в настоящем clickable.

Используем его в A и B:

```
fun Modifier.inputB(logs: SnapshotStateList<String>): Modifier {
   return this.myClickable { logs.add("B click") }
}

fun Modifier.inputA(logs: SnapshotStateList<String>): Modifier {
   return this.myClickable { logs.add("A click") }
}
```

Проверяем

Все работает.

await*

Важно помнить, что когда мы создаем низкоуровневый обработчик жестов, нам необходимо самим ставить и проверять consumed статус. А чем более высокоуровневые механизмы используем, тем меньше надо об этом статусе заботиться.

На прошлом уроке мы делали еще одну версию обработчика кликов. В нем мы использовали не awaitPointerEvent, а чуть более высокоуровневые await* методы.

```
fun Modifier.myClickable(onClick: () -> Unit): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           awaitFirstDown()
           val lastUp = waitForUpOrCancellation()
           if (lastUp != null) onClick()
       }
   }
}
```

awaitFirstDown и waitForUpOrCancellation умеют проверять consumed статус, это можно увидеть в их исходниках.

А вот ставить статус нам придется самим.

```
fun Modifier.myClickable(onClick: () -> Unit): Modifier {
   return this.pointerInput(Unit) {
       awaitEachGesture {
           val firstDown = awaitFirstDown()
           firstDown.consume()
           val lastUp = waitForUpOrCancellation()
           lastUp?.consume()
           if (lastUp != null) onClick()
       }
   }
}
```

Теперь обработчик работает корректно.

На самом деле крайне мало инфы по этой теме. Очень легко тут ошибиться и сделать неправильно. Если делаете свой обработчик, и есть сомнения, как корректно работать с consumed статусом, то я рекомендую смотреть, как работают под капотом detect* методы. Мы их еще не проходили. Но если вы откроете их исходники, там практически все будет понятно, потому что это просто обертки над await* методами.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.